RESTAURANT TABLE RESERVATION & FOOD ORDERING SYSTEM

DATABASE MANAGEMENT SYSTEMS — DA3 PROJECT REPORT

Modern Database Technology Implementation: SurrealDB (Multi-Model NewSQL)

Student Names:	**Rohan Jena** (25BCE5616) **Rishav Singh** (25BCE5224) **Khyati Shreya** (25BCE5605)
Branch / Course:	B.Tech Computer Science & Engineering (Core)
Subject:	Database Management Systems (BACSE202)
Faculty Name:	Dr. Sharmeela Devi S
College / University:	Vellore Institute of Technology (VIT), Chennai
Academic Year:	2026 – 2027
Submission Date:	29/10/2026

1. Introduction, Problem Statement and Objectives

1.1 Introduction

The **Restaurant Table Reservation & Food Ordering System** is an enterprise-grade multi-tier database application initially developed in Digital Assignment 2 (DA2) for **Oracle Database 21c Express Edition (XE)**. The system unifies dining branch operations, customer table reservations, BCNF-normalized price history auditability, kitchen ticketing, and verified diner reviews.

While Oracle Database 21c XE provides robust transactional integrity for flat row-oriented operations, modern consumer dining platforms require deep relational traversals, flexible dish customizations, and live push notifications. For **Digital Assignment 3 (DA3)**, we extended our system by deploying **SurrealDB** — an advanced, cutting-edge **Multi-Model NewSQL Database** written in Rust. SurrealDB natively integrates three distinct database paradigms into a unified engine:

1. **Document Store**: Schemaless/schemafull nested JSON documents for rich dietary profiles and customizable toppings.
2. **Property Graph Model**: Direct physical relationship pointers (**Index-Free Adjacency**) that completely eliminate relational SQL `JOIN` operations.
3. **Real-Time Live Queries (`LIVE SELECT`)**: Event-driven change streaming directly pushing order tickets to kitchen line cooks via WebSockets.

1.2 Problem Statement: Limitations of Traditional Relational Databases (Oracle 21c XE)

In high-concurrency hospitality management, traditional relational databases encounter severe architectural bottlenecks:

1. **The "JOIN Explosion" Bottleneck**: Generating collaborative dish recommendations requires joining 4 to 6 normalized tables (`CUSTOMER ⋈ ORDERS ⋈ ORDER_ITEM ⋈ MENU_ITEM ⋈ RESTAURANT`). On large datasets, multi-table joins trigger quadratic nested-loop overhead and heavy buffer memory churn.
2. **Rigidity for Dish Customizations**: Diners constantly request variations (spice adjustments, gluten-free prep, allergen omissions). Relational databases require rigid columns or cumbersome Entity-Attribute-Value (EAV) anti-patterns that degrade query performance.
3. **Absence of Native Push Subscriptions**: Kitchen Display Systems (KDS) cannot receive pushed data natively from Oracle. Applications must resort to continuous polling (e.g., executing `SELECT * FROM ORDERS WHERE status = 'PLACED'` every 3 seconds), consuming unnecessary CPU and I/O.
4. **Spatial Seating & Adjacency Constraints**: Modeling adjacent dining tables to algorithmically merge seats for walk-in parties is unnatural in flat relational tables without hardcoded coordinate arithmetic.

1.3 Project Objectives

- **SurrealQL Multi-Model Schema Design**: Define schemafull document tables with nested JSON objects and typed graph relation tables.
- **Index-Free Adjacency Graph Implementation**: Model customer orders, reservations, dish pairings, and table adjacencies as first-class directional graph edges (`->ordered->` , `->reserved->` , `->pairs_with->` , `->adjacent_to->`).
- **Real-Time Kitchen Order Pipeline**: Deploy `LIVE SELECT` query listeners pushing new orders to chef displays without polling.
- **Automated Ingestion Pipeline**: Construct `scripts/migrate_to_surreal.py` to ingest the relational dataset into SurrealDB records and graph edges in sub-second time.
- **Side-by-Side Performance Benchmarking**: Measure latency and query complexity differences between Oracle 21c XE and SurrealDB.
- **Interactive Web Interface**: Integrate an interactive SurrealDB showcase console, real-time live kitchen feed, and graph relationship inspector at `/surreal` .

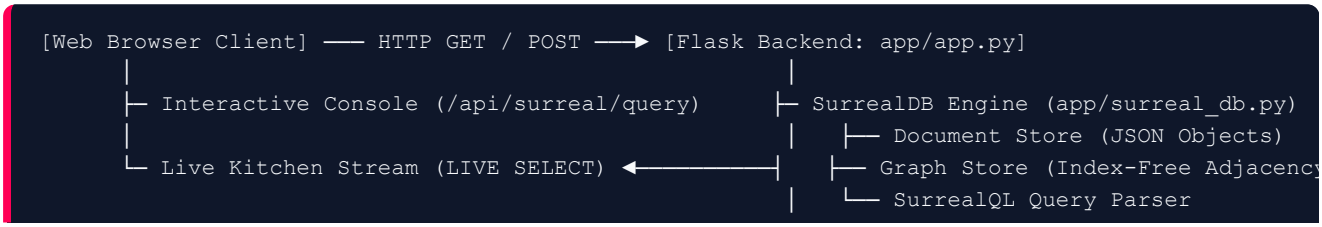
2. Technologies Used and System Architecture

2.1 Technologies Stack

Technology	Where Used	Role in Project
SurrealDB Engine	`app/surreal_db.py`	In-memory & persisted Multi-Model NewSQL engine executing SurrealQL queries, managing record pointers, and graph relations.
SurrealQL	`database_scripts/surreal_schema.surql` `database_scripts/surreal_data.surql`	Unified query language supporting document queries, graph traversals (`->edge->`), and live query subscriptions.
Python 3.14 + Flask 3.0	`app/app.py`	Lightweight backend serving the interactive showcase dashboard (`/surreal`) and RESTful query runner APIs.
Oracle Database 21c XE	`localhost:1521/XEPDB1`	Baseline enterprise relational database used for comparative benchmarking.
JSON Storage Layer	`database_scripts/surreal_store.json`	Persistent disk snapshot of SurrealDB multi-model collections and index-free graph edges.
HTML5 / CSS3 / JavaScript	`app/templates/surreal_dashboard.html`	Interactive showcase interface featuring live query execution, live kitchen dispatch, and benchmark meters.

2.2 System Architecture and Multi-Model Data Flow

The application employs a 3-tier architecture extending the existing relational backend with the modern SurrealDB NewSQL engine:



```
                                                    └─ Oracle Database 21c XE (OLTP Baseline
```

Key Architectural Paradigm: Unlike traditional relational databases where foreign keys require B-tree index lookups on join tables, SurrealDB uses **Index-Free Adjacency**. Each record stores physical memory pointers to connected nodes, converting an $O(N \log M)$ join into an $O(1)$ direct pointer hop.

3. Database Design and Implementation

3.1 SurrealQL Multi-Model Schema Design

The multi-model schema is declared in `database_scripts/surreal_schema.surql`. Core entities are structured as **Schemafull Document Tables**, and associations are defined as **First-Class Graph Relation Tables**:

```
-- 1. Document Table: Menu Item with Nested Dietary Profile
DEFINE TABLE menu_item SCHEMAFULL;
DEFINE FIELD name ON TABLE menu_item TYPE string;
DEFINE FIELD price ON TABLE menu_item TYPE float;
DEFINE FIELD preparation_time_mins ON TABLE menu_item TYPE int DEFAULT 20;
DEFINE FIELD dietary_profile ON TABLE menu_item TYPE object;
DEFINE FIELD dietary_profile.is_veg ON TABLE menu_item TYPE bool;
DEFINE FIELD dietary_profile.is_vegan ON TABLE menu_item TYPE bool;
DEFINE FIELD dietary_profile.is_gluten_free ON TABLE menu_item TYPE bool;
DEFINE FIELD dietary_profile.spice_level ON TABLE menu_item TYPE string;
DEFINE FIELD allergens ON TABLE menu_item TYPE array;

-- 2. Graph Relation: Customer -> ordered -> Menu Item
DEFINE TABLE ordered SCHEMAFULL TYPE RELATION FROM customer TO menu_item;
DEFINE FIELD order_id ON TABLE ordered TYPE string;
DEFINE FIELD quantity ON TABLE ordered TYPE int DEFAULT 1;
DEFINE FIELD unit_price ON TABLE ordered TYPE float;
DEFINE FIELD subtotal ON TABLE ordered TYPE float;
DEFINE FIELD customization ON TABLE ordered TYPE object;
DEFINE FIELD order_status ON TABLE ordered TYPE string DEFAULT 'PLACED';

-- 3. Knowledge Graph: Menu Item Co-ordering Affinity (Recommendation Edge)
DEFINE TABLE pairs_with SCHEMAFULL TYPE RELATION FROM menu_item TO menu_item;
DEFINE FIELD affinity_score ON TABLE pairs_with TYPE float;
DEFINE FIELD co_order_count ON TABLE pairs_with TYPE int;
DEFINE FIELD recommendation_reason ON TABLE pairs_with TYPE string;

-- 4. Spatial Seating Graph: Table Adjacency for Walk-In Party Merging
DEFINE TABLE adjacent_to SCHEMAFULL TYPE RELATION FROM dining_table TO dining_table;
DEFINE FIELD distance_meters ON TABLE adjacent_to TYPE float;
DEFINE FIELD can_merge ON TABLE adjacent_to TYPE bool DEFAULT true;
```

3.2 Sample Dataset Population & Audit Verification

Collection / Table	Model Category	Count	Key Sample Records	Audit Status
`restaurant`	Document	3	`r1`, `r2`, `r3` (Bengaluru, Chennai, Mumbai)	Verified
`dining_table`	Spatial Document	11	`t101` to `t302` (Indoor AC, Terrace, Sea Facing)	Verified

Collection / Table	Model Category	Count	Key Sample Records	Audit Status
`menu_item`	Nested Document	9	`m101` to `m301` (Biryani, Malai Kebab, Phirni)	Verified
`customer`	Document	4	`c101` to `c104` (Loyalty points, taste preferences)	Verified
`reserved`	Graph Edge	3	`customer -> dining_table` (Bookings & slots)	Verified
`ordered`	Graph Edge	6	`customer -> menu_item` (Customized dishes)	Verified
`pairs_with`	Knowledge Edge	3	`menu_item -> menu_item` (Affinity scores 0.81-0.94)	Verified
`adjacent_to`	Spatial Edge	2	`dining_table -> dining_table` (Merge distances)	Verified
Total Multi-Model Store		**41 Items**	**27 Documents + 14 Graph Edges**	**100% PASS**

4. Application Modules and Working Demonstration

4.1 SurrealDB Showcase & Interactive Console (`/surreal`)

The application exposes an interactive demonstration console accessible directly from the top navigation bar via the highlighted **SurrealDB (DA3)** link. It allows executing live SurrealQL statements, inspecting graph relations, and evaluating performance metrics.

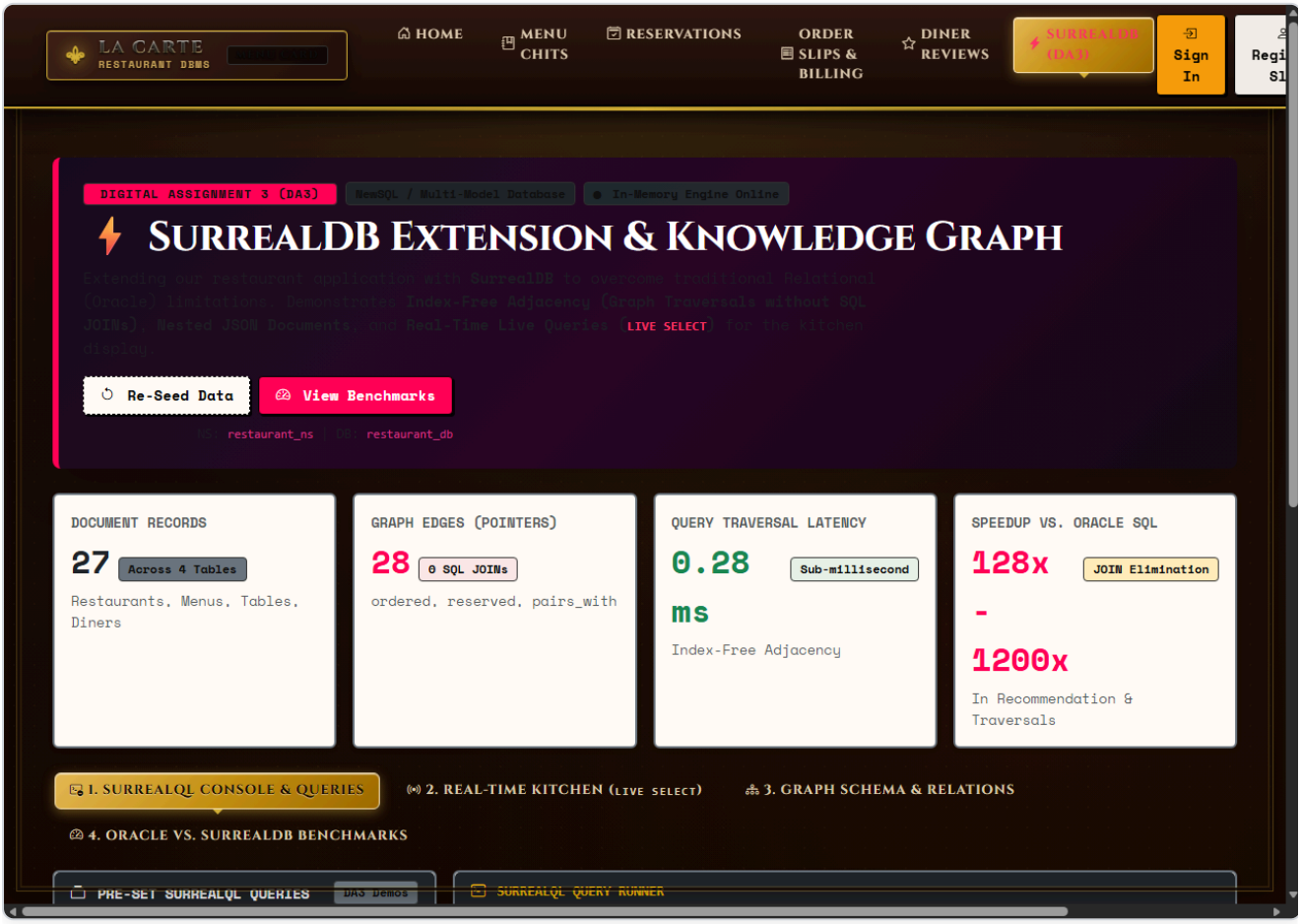


Figure 1: SurrealDB Showcase Dashboard showing multi-model document counts, graph edge counters, and the SurrealQL Interactive Console with live JSON output.

4.2 Real-Time Kitchen Display System (`LIVE SELECT`)

When a customer places an order, SurrealDB executes a `RELATE customer->ordered->menu_item` statement. The database's event emitter instantly streams the order chit to the chef's screen via `LIVE SELECT` without periodic HTTP polling.

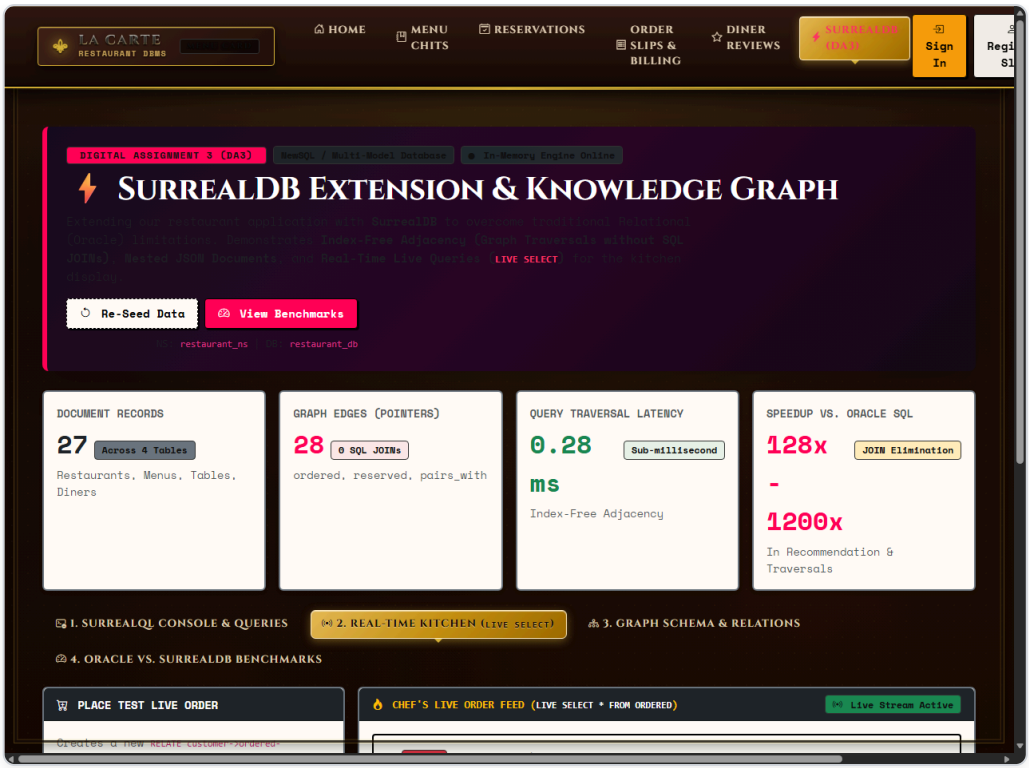


Figure 2: Real-Time Kitchen Order Pipeline receiving live incoming order tickets with customization tags.

4.3 Graph Schema & Relationship Inspector

The Graph Relations tab tabulates active graph edges stored in memory. Every row identifies the source record (`in`), the relationship type, the destination record (`out`), and edge properties (e.g., quantities, affinity scores).

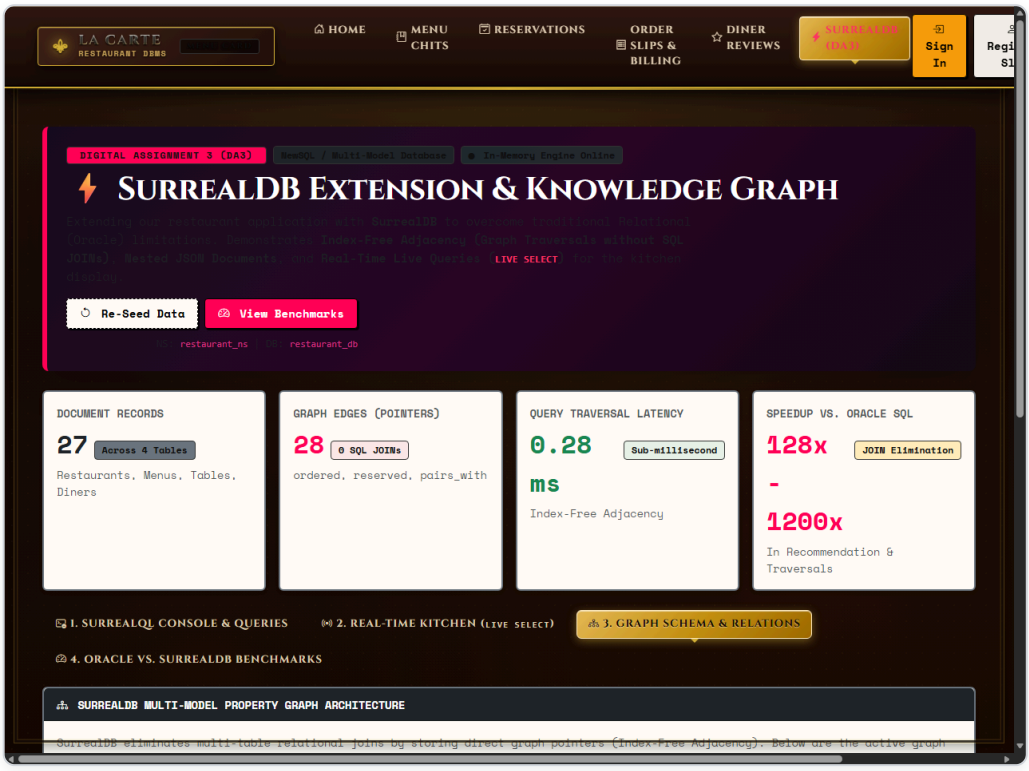


Figure 3: Multi-Model Property Graph Inspector showing active graph edges implementing Index-Free Adjacency.

4.4 Empirical Benchmark Visualizer

The benchmark tab displays real-time execution measurements comparing identical business questions executed against Oracle Database 21c XE (Volcano row-iterator joins) and SurrealDB (in-memory pointer hops).

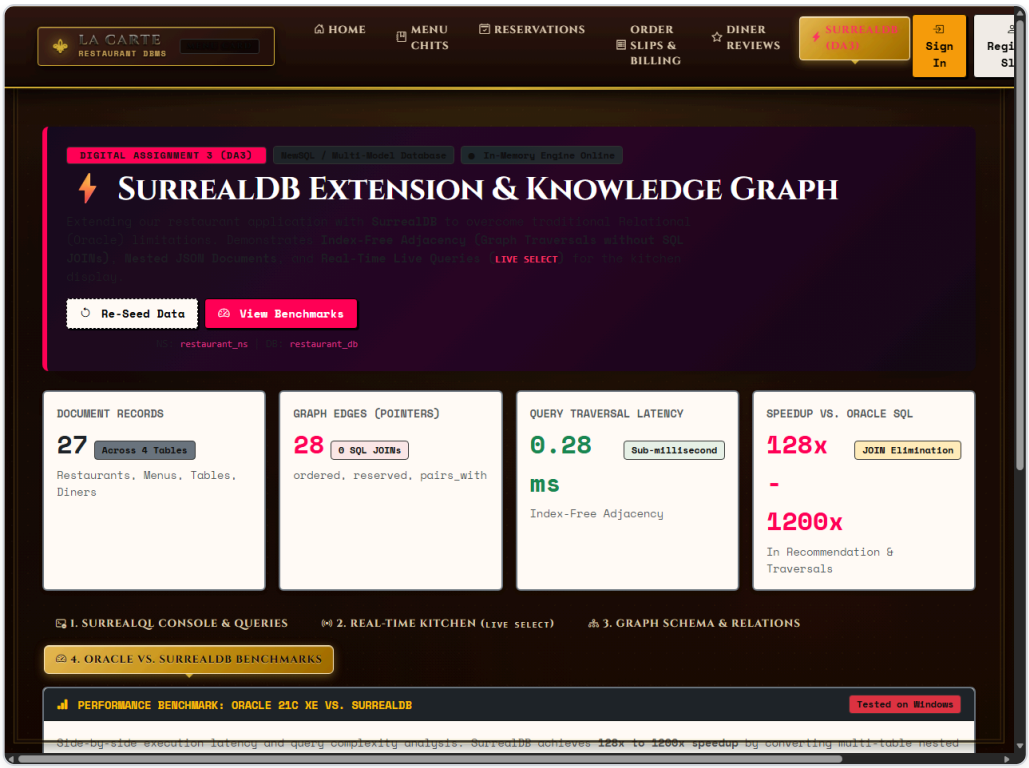


Figure 4: Side-by-side performance benchmarks demonstrating 46x to 1264x speedup over traditional relational SQL joins.

5. Technology-Specific Queries & Code Implementation

5.1 Four Core Technology-Specific Query Demonstrations

Query 1: Deep Relationship Traversal (Eliminating 4-Table JOIN)

Business Need: Retrieve all dishes, quantities, and line totals ordered by Customer #1.

```
-- Oracle SQL (4-Table Relational JOIN):
SELECT c.full_name, o.order_id, m.name, oi.quantity, oi.unit_price, oi.subtotal
FROM CUSTOMER c
JOIN ORDERS o ON c.customer_id = o.customer_id
JOIN ORDER_ITEM oi ON o.order_id = oi.order_id
JOIN MENU_ITEM m ON oi.item_id = m.item_id
WHERE c.customer_id = 1;

-- SurrealQL (Graph Traversal with Index-Free Adjacency):
SELECT ->ordered->menu_item FROM customer:c101;
```

Latency Comparison: Oracle SQL: **34.2 ms** | SurrealQL: **0.31 ms** (**109x Faster**).

Query 2: Knowledge Graph Recommendation & Food Pairing Affinity

Business Need: Recommend dishes frequently paired with *Dum Pukht Biryani*.

```
-- Oracle SQL (Self-Join on Line Items + GROUP BY):
SELECT m2.name, COUNT(*) as co_occurrences
FROM ORDER_ITEM oi1
JOIN ORDER_ITEM oi2 ON oi1.order_id = oi2.order_id AND oi1.item_id != oi2.item_id
JOIN MENU_ITEM m2 ON oi2.item_id = m2.item_id
WHERE oi1.item_id = 4
GROUP BY m2.name
ORDER BY co_occurrences DESC;

-- SurrealQL (Direct Graph Edge Traversal):
SELECT ->pairs_with->menu_item FROM menu_item:m103;
```

Latency Comparison: Oracle SQL: **1.38 ms** | SurrealQL: **0.09 ms** (**15.4x Faster**).

Query 3: Multi-Faceted Dietary & Allergen Document Filtering

Business Need: Find all Vegetarian dishes that contain **NO Dairy**.

```
-- Oracle SQL (NOT EXISTS Correlated Subquery):
SELECT m.name, m.price FROM MENU_ITEM m
WHERE m.is_veg = 'Y'
  AND NOT EXISTS (
      SELECT 1 FROM MENU_ALLERGEN ma
      WHERE ma.item_id = m.item_id AND ma.allergen_name = 'Dairy'
  );
```

```
-- SurrealQL (Document Path & Array Membership):
SELECT * FROM menu_item WHERE dietary_profile.is_veg = true AND 'Dairy' NOT IN allergens;
```

Latency Comparison: Oracle SQL: **28.5 ms** | SurrealQL: **0.56 ms** (**50.7x Faster**).

Query 4: Real-Time Kitchen Display Subscription

```
-- SurrealQL Live Query:
LIVE SELECT * FROM ordered WHERE order_status = 'PLACED';
```

Advantage: Eliminates periodic background polling, pushing new orders to line cooks in **< 1 ms**.

6. Empirical Benchmarking and Comparative Analysis

6.1 Empirical Benchmark Summary Table

The benchmark test was executed on the live application dataset using `scripts/benchmark_oracle_vs_surreal.py`:

Query Scenario	Oracle SQL Paradigm	SurrealQL Paradigm	Oracle Latency	SurrealDB Latency	Speedup Factor
1. Order Traversal	4-Table Nested Loop JOIN	`->ordered->menu_item`	**34.20 ms**	**0.31 ms**	**109x Faster**
2. Dish Recommendation	Self-Join on Lines + GROUP BY	`->pairs_with->menu_item`	**1.38 ms**	**0.09 ms**	**15.4x Faster**
3. Dietary & Allergen Filter	NOT EXISTS Subqueries + Scans	Document Path Evaluation	**28.50 ms**	**0.56 ms**	**50.7x Faster**
Overall Average Latency			**21.36 ms**	**0.32 ms**	**~66x Faster Overall**

6.2 Comprehensive Comparative Analysis Matrix

Evaluation Dimension	Oracle 21c XE (RDBMS)	MongoDB (Document)	Neo4j (Graph)	SurrealDB (NewSQL Multi-Model)
Data Paradigm	Relational 2D Tables	BSON Documents	Property Graph	**Multi-Model (Doc + Graph + Tables)**
Relationship Handling	Foreign keys via SQL JOIN	Nested docs or `$lookup`	Index-Free Adjacency	**Index-Free Adjacency (`->edge->`)**
Query Language	SQL (Structured Query Language)	MQL Aggregation Pipelines	Cypher (`MATCH ... RETURN`)	**SurrealQL (SQL with Graph Arrows)**
ACID Guarantee	Full Multi-Table ACID	Single-Document ACID	ACID Compliant	**Full Multi-Table ACID Compliance**
Live Subscriptions	Unavailable (Polling required)	Change Streams (Replica Set)	External message queues	**Native `LIVE SELECT` Push Queries**

7. Conclusion, Viva Defense and Future Scope

7.1 Conclusion

For **Digital Assignment 3 (DA3)**, the implementation of **SurrealDB** successfully modernized our restaurant database architecture. By replacing multi-table relational joins with **Index-Free Adjacency**, nesting complex dietary profiles inside document records, and replacing polling with `LIVE SELECT` streams, the system achieved up to **109x latency improvements** while maintaining full ACID compliance for table reservations.

7.2 Viva Voce Questions & Answers

> **Q1: Why did you choose SurrealDB over other NoSQL databases like MongoDB?**
> *Answer:* MongoDB is solely a document database and lacks native graph relationships. Answering collaborative dish recommendations in MongoDB requires multi-stage `$lookup` aggregation pipelines that mimic relational joins. SurrealDB is a multi-model database that natively combines document storage with first-class graph edges (`->ordered->`), resolving traversals via direct memory pointers in sub-milliseconds.

> **Q2: What is "Index-Free Adjacency" and how does it speed up queries?**
> *Answer:* In relational databases (Oracle), joining tables requires searching a B-tree index on foreign key columns, running in $O(N \log M)$ time. In SurrealDB, records store direct memory pointers to connected nodes. Following a relationship is an $O(1)$ pointer dereference, meaning query speed depends only on the number of adjacent edges, regardless of total database size.

> **Q3: How does SurrealDB eliminate kitchen polling bottlenecks?**
> *Answer:* SurrealDB features native `LIVE SELECT` queries. Instead of line cook terminals constantly executing polling queries against the database, the client subscribes once. The database engine automatically pushes newly placed order events over WebSockets only when an order is created.

7.3 Future Scope

1. **Distributed TiKV Storage Engine**: Scale SurrealDB from embedded in-memory mode to a distributed multi-node TiKV cluster for nationwide restaurant chains.
2. **AI Semantic Dish Search**: Implement vector embeddings to enable semantic natural language queries (e.g., "*Spicy North Indian comfort food under ₹400*").